

```
import sqlite3
```

```
SCHEMA = ""
```

```
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE departments (  
    department_id INTEGER PRIMARY KEY,  
    department_name TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE employees (  
    employee_id INTEGER PRIMARY KEY,  
    employee_name TEXT NOT NULL,  
    department_id INTEGER NOT NULL REFERENCES departments(department_id)  
);
```

```
CREATE TABLE shifts (  
    shift_id INTEGER PRIMARY KEY,  
    shift_name TEXT NOT NULL UNIQUE,  
    start_hour INTEGER NOT NULL CHECK(start_hour BETWEEN 0 AND 23),  
    end_hour INTEGER NOT NULL CHECK(end_hour BETWEEN 0 AND 23),  
    CHECK(start_hour != end_hour)  
);
```

```
CREATE TABLE shift_assignments (  
    assignment_id INTEGER PRIMARY KEY,  
    employee_id INTEGER NOT NULL REFERENCES employees(employee_id),  
    work_date TEXT NOT NULL,
```

```

        shift_id INTEGER NOT NULL REFERENCES shifts(shift_id),
        UNIQUE(employee_id, work_date)
    );
CREATE TABLE attendance (
    attendance_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL UNIQUE
        REFERENCES shift_assignments(assignment_id),
    check_in TEXT NOT NULL,
    check_out TEXT NOT NULL CHECK(check_out > check_in)
);
CREATE TABLE leave_requests (
    leave_id INTEGER PRIMARY KEY,
    employee_id INTEGER NOT NULL REFERENCES employees(employee_id),
    start_date TEXT NOT NULL,
    end_date TEXT NOT NULL CHECK(end_date >= start_date),
    reason TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'Pending'
        CHECK(status IN ('Pending', 'Approved', 'Rejected'))
);

-- Approved leave prevents attendance for the same shift START date.
CREATE TRIGGER block_attendance_during_leave
BEFORE INSERT ON attendance
WHEN EXISTS (
    SELECT 1 FROM shift_assignments a
    JOIN leave_requests l ON a.employee_id = l.employee_id

```

```

WHERE a.assignment_id = NEW.assignment_id
      AND l.status = 'Approved'
      AND a.work_date BETWEEN l.start_date AND l.end_date
)
BEGIN
    SELECT RAISE(ABORT, 'Attendance blocked: employee is on approved leave.');
```

END;

-- Approval must also be refused if attendance already exists.

```

CREATE TRIGGER block_conflicting_leave_approval
BEFORE UPDATE OF status ON leave_requests
WHEN NEW.status = 'Approved' AND EXISTS (
    SELECT 1 FROM attendance t
    JOIN shift_assignments a ON a.assignment_id = t.assignment_id
    WHERE a.employee_id = NEW.employee_id
          AND a.work_date BETWEEN NEW.start_date AND NEW.end_date
)
BEGIN
    SELECT RAISE(ABORT, 'Leave approval blocked: attendance already exists.');
```

END;

-- Night shifts end on the day after their work_date.

```

CREATE VIEW schedule AS
SELECT a.*, s.shift_name,
       datetime(a.work_date, '+' || s.start_hour || ' hours') AS shift_start,
       datetime(a.work_date,
```

```
        CASE WHEN s.end_hour < s.start_hour THEN '+1 day' ELSE '+0 days' END,
        '+' || s.end_hour || ' hours') AS shift_end
FROM shift_assignments a JOIN shifts s ON a.shift_id = s.shift_id;
```

-- The simulation reports completed historical sample shifts as of Oct 8.

```
CREATE VIEW attendance_report AS
```

```
SELECT e.employee_id, e.employee_name, s.work_date, s.shift_name,
CASE
    WHEN EXISTS (
        SELECT 1 FROM leave_requests l WHERE l.employee_id = e.employee_id
            AND l.status = 'Approved'
            AND s.work_date BETWEEN l.start_date AND l.end_date
    ) THEN 'On Leave'
    WHEN t.check_in IS NOT NULL THEN
        CASE WHEN t.check_in > datetime(s.shift_start, '+10 minutes')
            THEN 'Late' ELSE 'Present' END
    WHEN s.shift_end <= '2026-10-08 08:00:00' THEN 'Absent'
    ELSE 'Scheduled'
END AS status,
COALESCE(ROUND((strftime('%s', t.check_out) - strftime('%s', t.check_in))
    / 3600.0, 2), 0) AS worked_hours
FROM schedule s
JOIN employees e ON s.employee_id = e.employee_id
LEFT JOIN attendance t ON s.assignment_id = t.assignment_id;
"""
```

```

def show(db, title, query):
    print('\n' + title)
    cursor = db.execute(query)
    headers = [column[0] for column in cursor.description]
    rows = [[str(value) for value in row] for row in cursor.fetchall()]
    widths = [
        max([len(h)] + [len(r[i]) for r in rows])
        for i, h in enumerate(headers)
    ]
    print(' | '.join(h.ljust(w) for h, w in zip(headers, widths)))
    print('-+-'.join('-' * w for w in widths))
    for row in rows:
        print(' | '.join(value.ljust(w) for value, w in zip(row, widths)))

def simulate():
    db = sqlite3.connect(':memory:')
    try:
        print('EMPLOYEE ATTENDANCE, SHIFT AND LEAVE MANAGEMENT SYSTEM')
        print('Automatic Python + SQL simulation | Fictional sample records')
        print('Report evaluation time: 2026-10-08 08:00:00')
        db.executescript(SCHEMA)
        print('\nSTEP 1: Created six related tables, two views and two triggers.')

        # Python supplies values; SQL inserts them in one transaction.

```

with db:

```
db.executemany('INSERT INTO departments VALUES (?, ?)',
               [(1, 'IT'), (2, 'HR'), (3, 'Operations')])
db.executemany('INSERT INTO employees VALUES (?, ?, ?)',
               [(1, 'Ananya Rao', 1), (2, 'Ravi Kumar', 2), (3, 'Sara Ali', 3)])
db.executemany('INSERT INTO shifts VALUES (?, ?, ?, ?)',
               [(1, 'Day', 9, 17), (2, 'Evening', 14, 22), (3, 'Night', 22, 6)])
# Employee 1 and 2 work days; employee 3 works nights.
db.executemany('INSERT INTO shift_assignments VALUES (?, ?, ?, ?)',
               [(1, 1, '2026-10-05', 1), (2, 2, '2026-10-05', 1),
                (3, 3, '2026-10-05', 3), (4, 1, '2026-10-06', 1),
                (5, 2, '2026-10-06', 1), (6, 3, '2026-10-06', 3)])
```

show(db, 'STEP 2: Employees and departments', '')

```
SELECT e.employee_id AS ID, e.employee_name AS Employee,
       d.department_name AS Department
FROM employees e JOIN departments d ON e.department_id = d.department_id
ORDER BY e.employee_id''')
```

show(db, 'STEP 3: Assigned shifts', '')

```
SELECT e.employee_name AS Employee, s.work_date AS Date,
       s.shift_name AS Shift, s.shift_start AS Start, s.shift_end AS End
FROM schedule s JOIN employees e ON s.employee_id = e.employee_id
ORDER BY s.work_date, e.employee_id''')
```

with db:

```
db.executemany(''INSERT INTO leave_requests
```

```

        (leave_id, employee_id, start_date, end_date, reason)
VALUES (?, ?, ?, ?, ?)''' ,
        [(1, 1, '2026-10-06', '2026-10-06', 'Medical rest'),
         (2, 2, '2026-10-06', '2026-10-06', 'Personal work'),
         (3, 3, '2026-10-07', '2026-10-07', 'Family event')])
db.execute(
    'UPDATE leave_requests SET status = ? WHERE leave_id = ?',
    ('Approved', 1))
db.execute(
    'UPDATE leave_requests SET status = ? WHERE leave_id = ?',
    ('Rejected', 2))

show(db, 'STEP 4: Leave requests and administrator decisions', '''
SELECT e.employee_name AS Employee, l.start_date AS Start,
       l.end_date AS End, l.reason AS Reason, l.status AS Status
FROM leave_requests l JOIN employees e ON l.employee_id = e.employee_id
ORDER BY l.leave_id''')

with db:
    db.executemany('''INSERT INTO attendance
        (assignment_id, check_in, check_out) VALUES (?, ?, ?)''' ,
        [(1, '2026-10-05 09:00:00', '2026-10-05 17:00:00'),
         (2, '2026-10-05 09:20:00', '2026-10-05 17:00:00'),
         (3, '2026-10-05 22:00:00', '2026-10-06 06:00:00'),
         (6, '2026-10-06 22:00:00', '2026-10-07 06:00:00')])
print('\nSTEP 5: Recorded check-in and check-out times for four shifts.')

```

```

show(db, 'STEP 6: Daily attendance report', '''
    SELECT work_date AS Date, employee_name AS Employee,
           shift_name AS Shift,
           status AS Status, worked_hours AS Hours FROM attendance_report
    ORDER BY work_date, employee_id''')
show(db, 'STEP 7: Employee summary for the two sample work dates', '''
    SELECT employee_name AS Employee, COUNT(*) AS Assigned,
           SUM(status IN ('Present', 'Late')) AS Attended,
           SUM(status = 'Late') AS Late, SUM(status = 'Absent') AS Absent,
           SUM(status = 'On Leave') AS Leave,
           ROUND(SUM(worked_hours), 2) AS Hours
    FROM attendance_report GROUP BY employee_id, employee_name
    ORDER BY employee_id''')

print('\nSTEP 8: Demonstrate database validation and rollback')
for assignment, start, end, label in [
    (4, '2026-10-06 09:00:00', '2026-10-06 17:00:00',
     'Attendance during approved leave'),
    (1, '2026-10-05 09:00:00', '2026-10-05 17:00:00',
     'Duplicate attendance')
]:
    try:
        with db:
            db.execute('''INSERT INTO attendance
                           (assignment_id, check_in, check_out) VALUES (?, ?, ?)''',

```

```
        (assignment, start, end))
except sqlite3.IntegrityError:
    print(label + ': correctly blocked; transaction rolled back.')
else:
    raise AssertionError('Expected database validation did not run.')

assert db.execute('SELECT COUNT(*) FROM attendance').fetchone()[0] == 4
assert db.execute('PRAGMA integrity_check').fetchone()[0] == 'ok'
assert db.execute('PRAGMA foreign_key_check').fetchall() == []
print('\nSIMULATION COMPLETED SUCCESSFULLY.')
print('Present/Late: based on a 10-minute arrival grace period.')
print('Night shifts and leave use the shift START date.')
print('Late is included in Attended; recorded hours do not deduct breaks.')
print('Temporary database: run the program again to repeat the demo.')
finally:
    db.close()
```

```
if __name__ == '__main__':
    simulate()
```